

## Strukturalni paterni

### Analiza strukturalnih obrazaca i primjena na sistem "BentoLab"

#### 1. Adapter (Adapter)

- **Opis:** Omogućava objektima sa nekompatibilnim interfejsima da sarađuju.
- **Primjena:** Naš sistem koristi klasu Obavještenje. Ukoliko u budućnosti budemo koristili vanjski servis za slanje e-mailova (npr. *SendGrid*), koji ima drugačije nazive metoda od naših, kreirali bismo EmailAdapter. On bi preveo našu internu logiku u format koji vanjski servis razumije.

#### 2. Facade (Fasada)

- **Opis:** Pruža unificirani i jednostavan interfejs za skup interfejsa u jednom podsistemu.
- **Primjena:** Proces naručivanja torte je kompleksan jer uključuje provjeru dostupnosti, obračun cijene, stavke narudžbe i dostavu. Uvođenjem klase NarudzbaFacade, klijent (kontroler) poziva samo jednu metodu ZavršiNarudžbu(), dok fasada interno upravlja objektima Narudžba, Dostava i StavkaNarudžbe.

#### 3. Decorator (Dekorator)

- **Opis:** Omogućava dinamičko dodavanje funkcionalnosti objektu bez izmjene njegove strukture.
- **Primjena:** Na osnovnu tortu korisnik može dodati "ekstra opcije" poput posebnog pakovanja ili brze isporuke. Svaka opcija bi bila jedan dekorator koji "umotava" osnovni objekat Torta i dodaje cijenu na osnovnu vrijednost.

#### 4. Bridge (Most)

- **Opis:** Razdvaja apstrakciju od njezine implementacije, omogućavajući im da se mijenjaju nezavisno.
- **Primjena:** Klasa Dostava se može realizovati na različite načine (npr. *VlastitaDostava* i *EksternaKurirskaSlužba*). Most omogućava da logika obračuna dostave bude odvojena od tehničke implementacije praćenja pošiljke.

#### 5. Composite (Kompozit)

- **Opis:** Omogućava klijentima da tretiraju pojedinačne objekte i kompozicije objekata na isti način.

- **Primjena:** Ako slastičarna uvede "Party setove" (koji se sastoje od više bento torti i dodatnih ukrasa), setovi bi se tretirali kao jedan objekat. I jedna Torta i PartySet bi implementirali isti interfejs sa metodom `getFinalnaCijena()`.

## 6. Proxy (Proksi)

- **Opis:** Pruža zamjenski objekat koji kontroliše pristup originalnom objektu.
- **Primjena:** Može se primijeniti za klasu Dekoracija. Ako sistem sadrži hiljade slika visoke rezolucije za dekoracije, *Virtual Proxy* bi mogao učitavati slike tek u trenutku kada korisnik klikne na određenu dekoraciju, čime se štede resursi sistema.

## 2. Implementacija odabranih obrazaca

Za nadogradnju prvobitnog klasnog dijagrama odabrani su obrasci **Facade** i **Adapter** jer direktno rješavaju problem kompleksnosti procesa kupovine i integracije sa vanjskim servisima.

### Novi Klasni Dijagram (Opis izmjena)

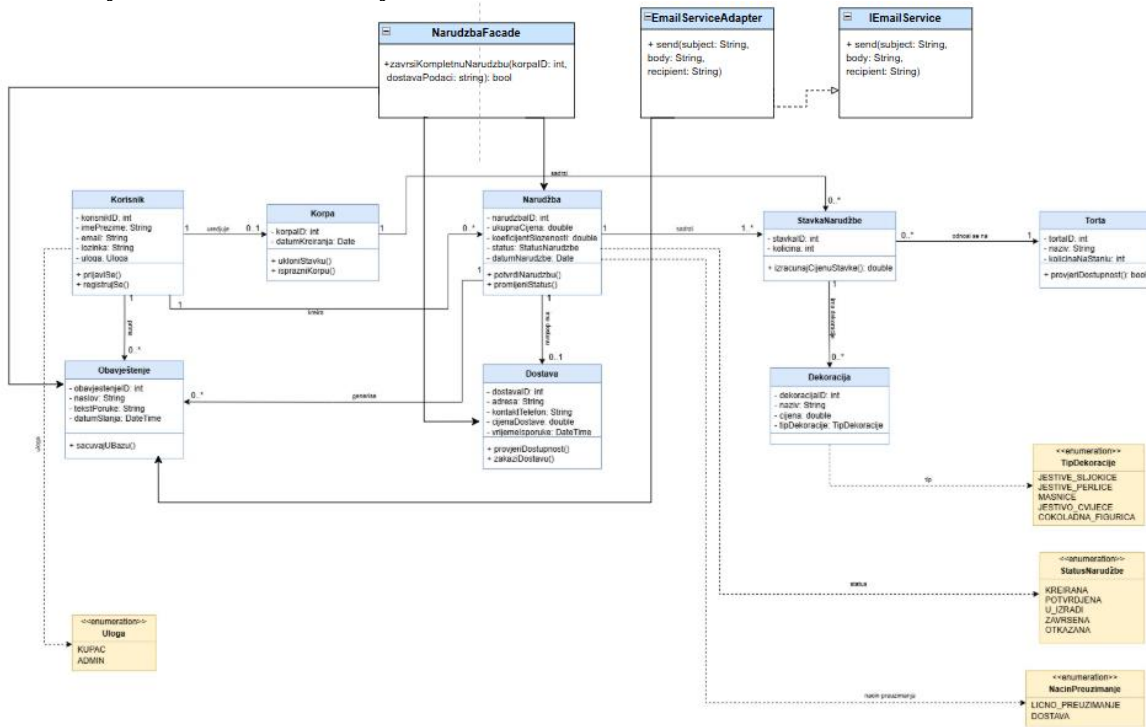
Na osnovni klasni dijagram dodani su sljedeći elementi:

#### 1. **NarudzbaFacade (Klasa):** Postavlja se između korisničkog interfejsa i poslovne logike

- **Metoda:** `izvrsiKompletnuNarudzbu(korpaID: int, dostavaPodaci: string): bool`
- **Veze:** Povlače se asocijacije (strelice) prema klasama **Narudžba**, **Dostava** i **Obavještenje**.

#### 2. **EmailAdapter (Klasa):**

- Dodaje se pored klase **Obavještenje**.
- **Metoda:** `send(subject, body, recipient)`
- **Veze:** Implementira interfejs **IService** i koristi klasu **Obavještenje** za dobavljanje podataka o poruci.



- **NarudzbaFacade** je uveden kako bi se sakrila kompleksnost procesa kreiranja narudžbe, objedinjujući rad s korpom, kreiranje same narudžbe i zakazivanje dostave.
- **EmailServiceAdapter** implementira IEmailService interfejs, omogućavajući sistemu da šalje obavještenja putem vanjskog servisa bez direktne ovisnosti o konkretnoj implementaciji slanja.